Graph Representation

Lecture : Fundamentals, Terminology, Adjacency Matrix, Adjacency List and Adjacency Multi-List

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

What Is a Graph?

Definition

A graph is a non-linear data structure consisting of a set of vertices and a set of edges connecting those vertices.

Mathematically:

$$\boxed{G = (V, E)}$$

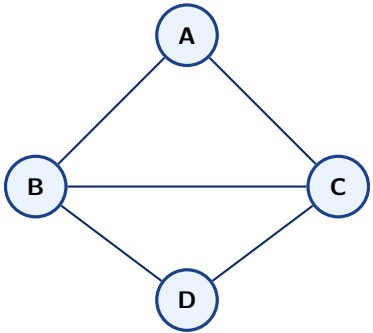
where:

- V represents the set of vertices.
- E represents the set of edges.

Example

A social network can be represented as a graph where users are vertices and relationships are edges.

Graph: A Simple Example



$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (A, C), (B, C), (B, D), (C, D)\}$$

Why Do We Need Graphs?

Many real-world problems involve relationships.

- Social networks.
- Road and transportation networks.
- Computer communication networks.
- Web page hyperlinks.
- Airline routes.
- Dependency relationships.
- Electrical circuits.
- Recommendation systems.

Key Idea

Graphs represent connections between objects, rather than only storing objects sequentially.

Linear vs Non-Linear Data Structures

Linear Structures	Non-Linear Structures
Arrays	Trees
Linked lists	Graphs
Stacks	Hierarchical structures
Queues	Network structures

Observation

A graph can represent arbitrary relationships and may contain cycles.

Vertices and Edges

Vertex

A vertex, also called a node, represents an object or entity.

Edge

An edge represents a relationship between two vertices.

For an undirected graph:

$$(A, B) = (B, A)$$

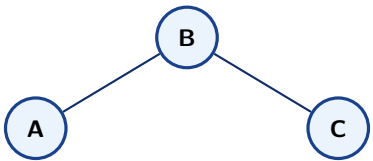
For a directed graph:

$$(A, B) \neq (B, A)$$

in general.

Adjacent Vertices

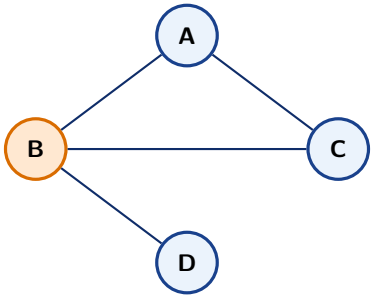
Two vertices are adjacent if an edge connects them.



- A and B are adjacent.
- B and C are adjacent.
- A and C are not adjacent.

Incident Edges

An edge is incident on a vertex if the vertex is one of its endpoints.



Observation

Three edges are incident on vertex B.

Degree of a Vertex

Definition

The degree of a vertex in an undirected graph is the number of edge ends incident on it. A self-loop contributes two.

Notation:

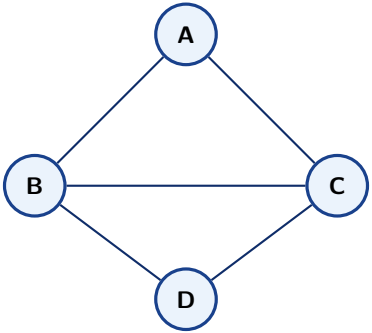
$$\deg(v)$$

Example

If vertex A is connected to B, C and D:

$$\boxed{\deg(A) = 3}$$

Degree Example



$$\deg(A) = 2, \quad \deg(B) = 3$$

$$\deg(C) = 3, \quad \deg(D) = 2$$

$$\sum_{v \in V} \deg(v) = 10 = 2|E|$$

Handshaking Lemma

For any finite undirected graph:

$$\boxed{\sum_{v \in V} \deg(v) = 2|E|}$$

Explanation

Every edge contributes two to the total degree count.

Example

If a graph has seven edges:

$$\sum \deg(v) = 14$$

In-Degree and Out-Degree

For directed graphs:

In-Degree

Number of edges entering a vertex.

$$\deg^-(v)$$

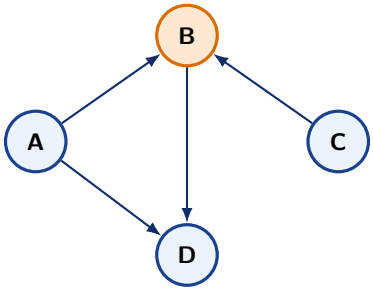
Out-Degree

Number of edges leaving a vertex.

$$\deg^+(v)$$

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|$$

Directed Degree Example



For vertex B:

$$\boxed{\deg^-(B) = 2}$$

$$\boxed{\deg^+(B) = 1}$$

Walk, Trail and Path

Walk

A sequence of vertices where consecutive vertices are connected by edges. Vertices and edges may repeat.

Trail

A walk in which no edge is repeated.

Simple Path

A path in which no vertex is repeated.

Example

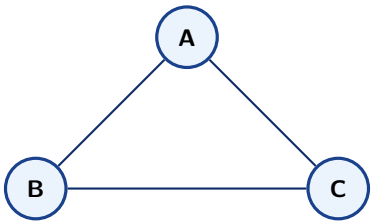
$$A \to B \to C \to D$$

is a simple path if all four vertices are distinct and consecutive edges exist.

Cycle

Definition

A cycle is a closed path that starts and ends at the same vertex, without repeating intermediate vertices.

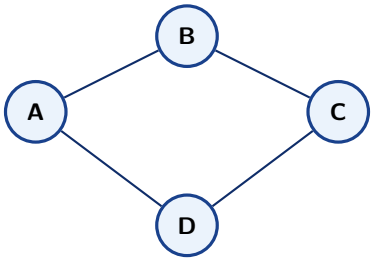


$$A \to B \to C \to A$$

Connected Graph

Definition

An undirected graph is connected if a path exists between every pair of vertices.



Every vertex is reachable from every other vertex.

Disconnected Graph



Observation

This graph has two connected components.
No path connects A to D.

Connected Components

Definition

A connected component is a maximal connected subgraph of an undirected graph.

Example

If:

$$V = \{A, B, C, D, E, F\}$$

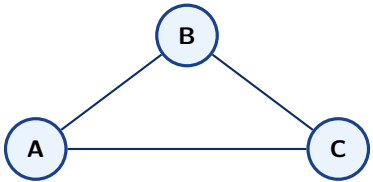
and the components are:

$$\{A, B, C\}, \quad \{D, E\}, \quad \{F\}$$

then the graph has three components.

Undirected Graph

In an undirected graph, edges do not have a direction.



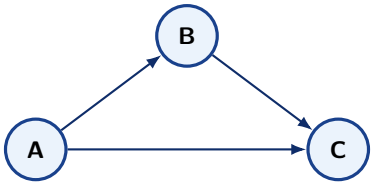
$$(A, B) = (B, A)$$

Example

A mutual friendship relationship.

Directed Graph

In a directed graph, edges have a specified direction.



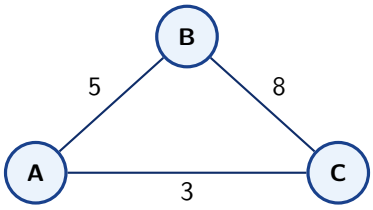
$$A \to B$$

does not imply:

$$B \to A$$

Weighted Graph

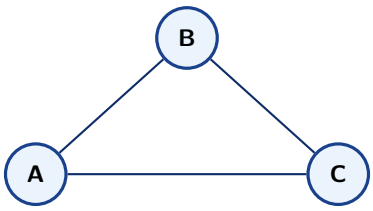
A weighted graph assigns a value to each edge.



- Weights may represent distance.
- Weights may represent cost.
- Weights may represent time.

Unweighted Graph

In an unweighted graph, edges have no explicit numerical weights.



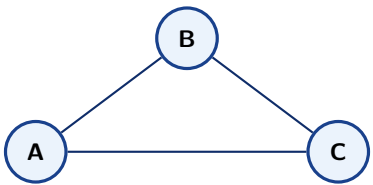
Example

A simple network showing whether two devices are connected.

Simple Graph

Definition

A simple undirected graph contains no self-loops and no parallel edges.



Multigraph

A multigraph permits multiple edges between the same vertices.



Example

Several different transport routes connecting the same two cities.

Complete Graph

A complete graph contains an edge between every pair of distinct vertices.
Notation:

$$K_n$$

For a simple undirected complete graph:

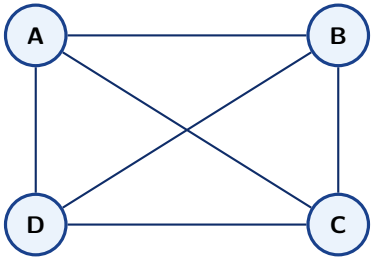
$$\boxed{|E| = \frac{n(n-1)}{2}}$$

Example

For $n = 5$:

$$|E| = \frac{5(4)}{2} = 10$$

Complete Graph: K_4



$$|V| = 4, \qquad |E| = 6$$

Cyclic and Acyclic Graphs

Cyclic Graph

Contains at least one cycle.

Acyclic Graph

Contains no cycles.

Directed Acyclic Graph (DAG)

A directed graph with no directed cycle.
Commonly used in task scheduling and dependency modeling.

Graph Types: Comparison

Graph Type	Main Characteristic
Undirected	Edges have no direction
Directed	Edges have direction
Weighted	Edges have numerical values
Unweighted	No explicit edge weights
Simple	No self-loops or parallel edges
Multigraph	Parallel edges allowed
Complete	Every pair of vertices connected
Connected	Every pair is mutually reachable
Disconnected	More than one connected component
DAG	Directed and acyclic

Why Represent Graphs in Memory?

A mathematical graph must be stored using suitable data structures.
Common representations:

1. Adjacency matrix.
2. Adjacency list.
3. Adjacency multi-list.

Selection Criteria

- Number of vertices.
- Number of edges.
- Directed or undirected graph.
- Memory availability.
- Required operations.

What Is an Adjacency Matrix?

Definition

An adjacency matrix is a two-dimensional array used to represent edges between vertices.

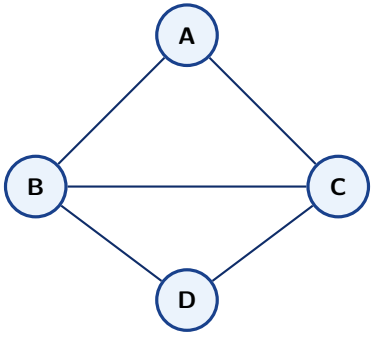
For n vertices:

$$A[n][n]$$

For an unweighted simple graph:

$$A[i][j] = \begin{cases} 1, & \text{if an edge exists} \\ 0, & \text{otherwise} \end{cases}$$

Adjacency Matrix: Example Graph



$$V = \{A, B, C, D\}$$

$$E = \{AB, AC, BC, BD, CD\}$$

Constructing the Adjacency Matrix

For the previous graph:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Adjacency Matrix: Step-by-Step

1. List vertices in a fixed order.
2. Create an $n \times n$ matrix.
3. Initialize all entries to zero.
4. For every edge (u, v), set:
$$A[u][v] = 1$$
5. For an undirected edge, also set:
$$A[v][u] = 1$$

Observation

The matrix of a simple undirected graph is symmetric.

Symmetry of an Undirected Graph

For an undirected graph:

$$A[i][j] = A[j][i]$$

Therefore:

$$\boxed{A = A^T}$$

Example

If A is connected to B:

$$A[A][B] = 1$$

and:

$$A[B][A] = 1$$

Adjacency Matrix for Directed Graphs

Consider:

$$A \to B, \quad A \to C, \quad C \to B$$

The adjacency matrix is:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Observation

The matrix is not necessarily symmetric.
Rows represent outgoing edges. Columns represent incoming edges.

Weighted Adjacency Matrix

For weighted graphs, entries store edge weights rather than only 0 or 1.

$$W = \begin{bmatrix} 0 & 5 & 3 \\ 5 & 0 & 8 \\ 3 & 8 & 0 \end{bmatrix}$$

Interpretation

- Edge A–B has weight 5.
- Edge A–C has weight 3.
- Edge B–C has weight 8.

Important

For general weighted graphs, use a separate marker for missing edges because a valid edge may have weight zero.

Space Complexity of Adjacency Matrix

For V vertices, the matrix contains:

$$V \times V$$

entries.
Therefore:

$$\boxed{O(V^2)}$$

Example

For 1,000 vertices:

$$1000 \times 1000 = 1,000,000$$

matrix entries are required.

Advantages of Adjacency Matrix

1. Simple to understand.
2. Easy to implement.
3. Constant-time edge existence check.
4. Suitable for dense graphs.
5. Convenient for matrix-based algorithms.

$$\boxed{\text{Edge lookup} = O(1)}$$

Disadvantages of Adjacency Matrix

- Requires $O(V^2)$ memory.
- Wastes space for sparse graphs.
- Finding all neighbors takes $O(V)$.
- Adding a new vertex may require resizing.
- Large graphs may require substantial memory.

What Is an Adjacency List?

Definition

An adjacency list stores, for every vertex, a list of its adjacent vertices.

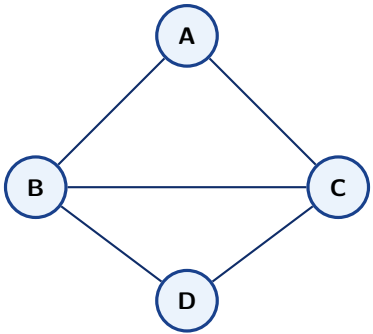
A common C implementation uses:

- An array of head pointers.
- Linked-list nodes for neighbors.

Benefit

Memory is proportional to the number of vertices and edges.

Adjacency List: Example Graph



We will represent the same graph using adjacency lists.

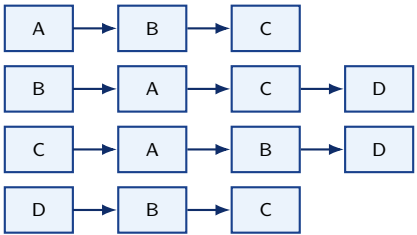
Adjacency List Representation

Vertex	Adjacent Vertices
A	B, C
B	A, C, D
C	A, B, D
D	B, C

Observation

Each undirected edge appears twice, once in each endpoint's list.

Adjacency List: Linked Representation



Adjacency List Construction

1. Create an array for all vertices.
2. Initialize each list head to NULL.
3. Read an edge (u, v).
4. Insert v into the list of u.
5. For an undirected graph, also insert u into the list of v.

Example

For edge (A, B):

$$A : B$$

$$B : A$$

Directed Graph Adjacency List

For a directed graph:

$$A \to B, \quad A \to C, \quad C \to B$$

the adjacency lists are:

Vertex	Outgoing Neighbors
A	B, C
B	NULL
C	B

Important

For directed graphs, only the source vertex receives the outgoing adjacency entry.

Space Complexity of Adjacency List

For V vertices and E edges:

$$\boxed{O(V + E)}$$

- V head pointers.
- E adjacency entries for directed graphs.
- $2E$ adjacency entries for undirected graphs.

Why Useful?

For sparse graphs where:

$$E \ll V^2$$

adjacency lists are usually more memory-efficient than matrices.

Advantages of Adjacency List

1. Efficient memory usage for sparse graphs.
2. Easy enumeration of neighbors.
3. Supports dynamic edge insertion.
4. Suitable for BFS and DFS.
5. Space complexity is $O(V + E)$.

Disadvantages of Adjacency List

- Edge existence checks may require list traversal.
- Pointer-based storage adds overhead.
- More complex implementation than matrices.
- Removing an edge may require searching a list.

Edge Lookup

With a basic linked adjacency list:

$$\boxed{O(\deg(u))}$$

for checking whether u has neighbor v.

What Is an Adjacency Multi-List?

Definition

An adjacency multi-list is a linked representation of an undirected graph in which each edge is stored in one shared edge record.

The edge record can be accessed from the adjacency chains of both endpoint vertices.

Motivation

Avoid storing two separate edge records for every undirected edge.

Adjacency Multi-List: Edge Record

A typical edge record contains:

$$\boxed{(\text{ivex}, \text{jvex}, \text{ilink}, \text{jlink})}$$

where:

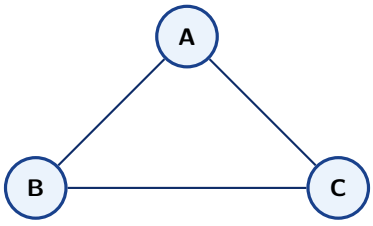
- `ivex`: first endpoint.
- `jvex`: second endpoint.
- `ilink`: next incident edge for the first endpoint.
- `jlink`: next incident edge for the second endpoint.

Additional fields may store edge weights or other information.

Adjacency Multi-List: Conceptual Example

Consider:

$$E = \{(A, B), (A, C), (B, C)\}$$



Edge Records

$$e_1 = (A, B), \quad e_2 = (A, C), \quad e_3 = (B, C)$$

Each edge record is shared by both endpoints.

Adjacency Multi-List: Structure

Edge Record	First Vertex	Second Vertex	Incident Vertices
e_1	A	B	A, B
e_2	A	C	A, C
e_3	B	C	B, C

Vertex Access

- A accesses e_1 and e_2.
- B accesses e_1 and e_3.
- C accesses e_2 and e_3.

Important

Only one edge record exists per undirected edge, even though it is reachable from two vertex chains.

Adjacency Multi-List: Advantages

1. Each undirected edge is represented once.
2. Both endpoints access the same edge record.
3. Edge information can be updated centrally.
4. Useful when edge records contain additional attributes.

Adjacency Multi-List: Limitations

- More complicated pointer management.
- Insertion and deletion require updating endpoint-specific links.
- Harder to implement than ordinary adjacency lists.
- Traversal must select the correct link based on the current endpoint.

C Program: Adjacency Matrix

We will implement an undirected graph using a two-dimensional array.

Operations

- Initialize the matrix.
- Insert an edge.
- Display the matrix.
- Check whether an edge exists.

C Program: Adjacency Matrix

```c
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int vertices;

void addEdge(int u, int v) {
    if (u < 0 || v < 0 ||
        u >= vertices || v >= vertices)
        return;

    graph[u][v] = 1;
    graph[v][u] = 1;
}

void displayMatrix(void) {
    printf("Adjacency Matrix:\n");

    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            printf("%d ", graph[i][j]);
        }
        printf("\n");
    }
}

int main(void) {
    vertices = 4;

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 2);
    addEdge(1, 3);
    addEdge(2, 3);
```

Adjacency Matrix: Program Output

Output

```
Adjacency Matrix:
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0
```

Vertex Mapping

$$0 = A, \quad 1 = B, \quad 2 = C, \quad 3 = D$$

Adjacency Matrix: Code Analysis

- The global matrix is initialized to zero.
- `addEdge()` updates two entries.
- The matrix is symmetric.
- `displayMatrix()` prints all entries.

Complexity

$$\text{Edge insertion} = O(1)$$

$$\text{Display} = O(V^2)$$

$$\text{Space} = O(V^2)$$

C Program: Adjacency List

We will use linked lists to represent an undirected graph.

Data Structures

- Array of adjacency-list heads.
- Linked-list nodes.
- Dynamic memory allocation.

Each undirected edge is inserted into both endpoint lists.

C Program: Node Structure

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

typedef struct Node {
    int vertex;
    struct Node *next;
} Node;

Node *adj[MAX];
int vertices;

Node *createNode(int v) {
    Node *p = malloc(sizeof(Node));

    if (p == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    p->vertex = v;
    p->next = NULL;

    return p;
}
```

C Program: Initialization and Insertion

```c
void initialize(int n) {
    vertices = n;

    for (int i = 0; i < vertices; i++)
        adj[i] = NULL;
}

void addEdge(int u, int v) {
    if (u < 0 || v < 0 ||
        u >= vertices || v >= vertices)
        return;

    Node *p = createNode(v);
    p->next = adj[u];
    adj[u] = p;

    Node *q = createNode(u);
    q->next = adj[v];
    adj[v] = q;
}
```

Note

This implementation inserts new neighbors at the beginning of each list.

C Program: Display and Cleanup

```c
void displayList(void) {
    for (int i = 0; i < vertices; i++) {
        printf("%d: ", i);

        Node *p = adj[i];

        while (p != NULL) {
            printf("%d -> ", p->vertex);
            p = p->next;
        }

        printf("NULL\n");
    }
}

void freeGraph(void) {
    for (int i = 0; i < vertices; i++) {
        Node *p = adj[i];

        while (p != NULL) {
            Node *temp = p;
            p = p->next;
            free(temp);
        }

        adj[i] = NULL;
    }
}
```

C Program: Main Function

```c
int main(void) {
    initialize(4);

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 2);
    addEdge(1, 3);
    addEdge(2, 3);

    displayList();

    freeGraph();

    return 0;
}
```

Adjacency List: Expected Output

Output

```
0:  2 -> 1 -> NULL
1:  3 -> 2 -> 0 -> NULL
2:  3 -> 1 -> 0 -> NULL
3:  2 -> 1 -> NULL
```

Observation

Neighbors appear in reverse insertion order because nodes are inserted at the list head.

Adjacency List: Code Analysis

- One head pointer is maintained per vertex.
- Each edge creates two list nodes.
- Dynamic memory stores adjacency entries.
- List traversal prints neighbors.

Complexity

$$\text{Space} = O(V + E)$$

$$\text{Display} = O(V + E)$$

$$\text{Edge insertion} = O(1)$$

for the head-insertion implementation.

Adjacency Matrix vs Adjacency List

Feature	Matrix	List
Storage	$O(V^2)$	$O(V + E)$
Edge lookup	$O(1)$	$O(\deg(u))$
Neighbor traversal	$O(V)$	$O(\deg(u))$
Edge insertion	$O(1)$	$O(1)$ with head insertion
Dense graphs	Suitable	May use many list nodes
Sparse graphs	Space inefficient	Space efficient

Dense vs Sparse Graphs

Dense Graph

A graph having a large number of edges relative to the maximum possible number.

Sparse Graph

A graph having relatively few edges.

For a simple undirected graph:

$$E_{\max} = \frac{V(V-1)}{2}$$

Practical Guideline

Dense graph: adjacency matrix.
Sparse graph: adjacency list.

Example: Memory Comparison

Suppose:

$$V = 1000, \qquad E = 2000$$

An adjacency matrix stores:

$$1000^2 = 1,000,000$$

entries.
An undirected adjacency list stores:

$$1000 \text{ heads} + 4000 \text{ neighbor entries}$$

Observation

The adjacency list requires far fewer logical entries for this sparse graph.

Actual memory usage also depends on integer sizes, pointer sizes and allocation overhead.

Which Representation Should We Choose?

Requirement	Preferred Representation
Fast edge existence check	Matrix
Sparse graph	List
Dense graph	Matrix
Enumerating neighbors	List
Simple implementation	Matrix
Shared undirected edge records	Multi-list

Worked Example 1

Question

Consider the undirected graph:

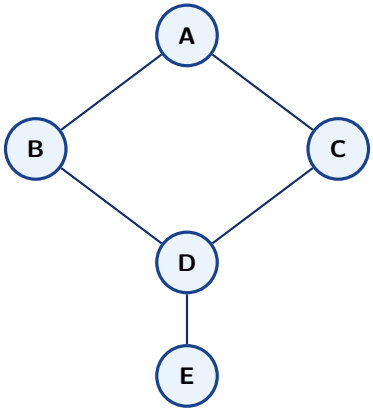
$$V = \{A, B, C, D, E\}$$

$$E = \{(A, B), (A, C), (B, D), (C, D), (D, E)\}$$

Construct:

1. Adjacency matrix.
2. Adjacency list.
3. Degree of every vertex.

Worked Example 1: Graph



Worked Example 1: Matrix

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Vertex order:

$$A,\ B,\ C,\ D,\ E$$

Observation

The matrix is symmetric.

Worked Example 1: List and Degree

Vertex	Adjacency List	Degree
A	B, C	2
B	A, D	2
C	A, D	2
D	B, C, E	3
E	D	1

$$\sum \deg(v) = 2 + 2 + 2 + 3 + 1 = 10$$

$$2|E| = 2(5) = 10$$

Handshaking lemma verified.

Worked Example 2: Directed Graph

Question

Given:

$$E = \{(A, B), (A, C), (B, C), (C, A), (C, D)\}$$

Find:

1. Adjacency matrix.
2. Adjacency list.
3. In-degree of each vertex.
4. Out-degree of each vertex.

Worked Example 2: Solution

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Vertex	In	Out	Outgoing Neighbors
A	1	2	B,C
B	1	1	C
C	2	2	A,D
D	1	0	None

$$\sum \deg^-(v) = 5$$

$$\sum \deg^+(v) = 5$$

Exercise 1

Question

An undirected graph contains six vertices and eight edges.
Calculate the sum of vertex degrees.

Solution

Using the handshaking lemma:

$$\sum \deg(v) = 2E$$

$$= 2(8) = \boxed{16}$$

Exercise 2

Question

How many edges are present in a complete undirected graph with eight vertices?

Solution

$$E = \frac{V(V-1)}{2}$$

$$E = \frac{8(7)}{2}$$

$$\boxed{E = 28}$$

Exercise 3

Question

A graph has:

$$V = 500, \qquad E = 600$$

Which representation is generally more memory-efficient?

Solution

Adjacency list.
Matrix entries:

$$500^2 = 250,000$$

Adjacency-list storage is proportional to:

$$V + E$$

rather than V^2.

Exercise 4: Matrix Construction

Question

Construct an adjacency matrix for:

$$V = \{A, B, C\}$$

$$E = \{(A, B), (B, C)\}$$

Assume an undirected graph.

Solution

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

MCQ 1

A graph is mathematically represented as:

1. $G = (A, B)$
2. $G = (V, E)$
3. $G = (X, Y, Z)$
4. $G = (N, P)$

Correct Answer

Option 2.

MCQ 2

The space complexity of an adjacency matrix is:

1. $O(V)$
2. $O(E)$
3. $O(V^2)$
4. $O(\log V)$

Correct Answer

Option 3.

MCQ 3

Which representation is generally preferred for sparse graphs?

1. Adjacency matrix
2. Adjacency list
3. Two-dimensional dense array
4. None of the above

Correct Answer

Option 2.

MCQ 4

For an undirected graph, the adjacency matrix is:

1. Always diagonal
2. Always triangular
3. Symmetric
4. Always an identity matrix

Correct Answer

Option 3.

MCQ 5

The sum of degrees of all vertices in an undirected graph equals:

1. E
2. $2E$
3. V
4. $V + E$

Correct Answer

Option 2.

MCQ 6

Which representation stores each undirected edge in one shared edge record?

1. Adjacency matrix
2. Standard adjacency list
3. Adjacency multi-list
4. Stack

Correct Answer

Option 3.

Important Graph Formulas

Graph Definition

$$\boxed{G = (V, E)}$$

Handshaking Lemma

$$\boxed{\sum_{v \in V} \deg(v) = 2E}$$

Complete Undirected Graph

$$\boxed{E = \frac{V(V-1)}{2}}$$

Representation Space

$$\boxed{\text{Matrix} : O(V^2)}$$

Lecture Summary

- Graphs consist of vertices and edges.
- Graphs represent relationships.
- Directed graphs contain oriented edges.
- Weighted graphs assign values to edges.
- Degree measures incident edges.
- Adjacency matrices use 2D arrays.
- Adjacency lists store neighbors.
- Adjacency multi-lists share edge records.
- Representation choice affects memory and operation efficiency.

Questions for Revision

1. Define a graph.
2. Explain degree, in-degree and out-degree.
3. Differentiate directed and undirected graphs.
4. Explain connected components.
5. Construct an adjacency matrix.
6. Construct an adjacency list.
7. Explain adjacency multi-lists.
8. Compare matrix and list representations.
9. Analyze graph storage complexity.
10. Explain practical graph applications.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 13: Graphs.
2. Thomas H. Cormen et al., *Introduction to Algorithms*, Graph Algorithms chapters.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Lecture Focus

Graph terminology and graph storage representations.

Next Lecture

Lecture : Breadth-First Search (BFS)

- Introduction to graph traversal.
- BFS working principle.
- Queue-based implementation.
- Step-by-step BFS dry runs.
- BFS traversal using adjacency lists.
- Complete C implementation.
- Time and space complexity.
- Applications and exercises.

Student Feedback

Share Your Feedback



Thank you!